

面向对象编程 II — 封装

面向对象编程 · 2026 · 南京大学

| 刘钦

知识点

- 封装的原则
- 构造方法的重载
- 对象的初步初始化
- 垃圾回收机制基本原理
- `static` 变量
- `static` 方法
- `static` 变量的初始化
- `final` 关键字

Outline

Part 1 — 封装

- Use case: GPS 位置类
- 五种设计的演进 ($A \rightarrow I \rightarrow II \rightarrow III$)
- 封装的三条规则
- 数据职责 + 行为职责

Part 2 — 类

- 类、对象、成员
- static 方法 vs 实例方法
- static 变量
- 静态变量初始化顺序

Part 2 (续)

- final 关键字 (变量/方法/类)
- final 参数 · 空白 final
- 构造方法
- 构造方法重载

Part 3 — 对象

- 对象的初始化顺序
- 引用比较 `==` vs `equals`
- 生存期 · 垃圾回收
- 分代回收机制

Part 1 — 封装

从结构化设计 → 真正的封装

Use Case: GPS 系统的位置类

Part of GPS system. **Latitude** and **Longitude** represent a **Position**. We want to compute the **distance** and **direction** between two positions.

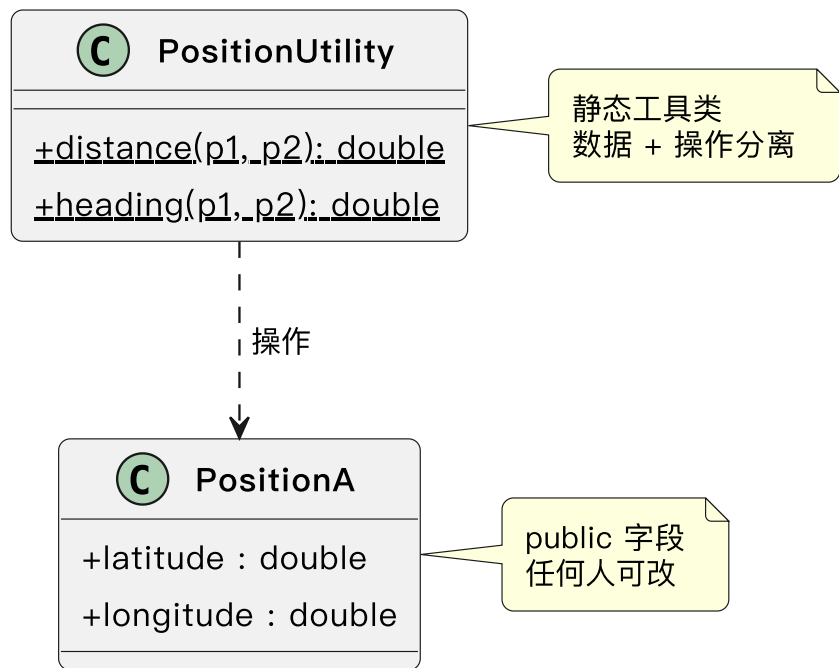
设计问题

- 如何用一个或多个类来实现这个用例？
- 数据放哪里？操作放哪里？
- 字段公开还是私有？
- 对象代表"一个点"还是"两个点"？

我们将给出 **5 种设计** (Design A, B, C, D, I, II, III), 逐步逼近真正的"封装"。

Design A — 数据与操作分离（结构化思维）

Design A — 数据与操作分离（结构化思维）



```
public class PositionA {
    public double latitude;
    public double longitude;
}

class PositionUtility {
    public static double distance(
        PositionA p1, PositionA p2) {
        return 0.0;
    }
    public static double heading(
        PositionA p1, PositionA p2) {
        return 0.0;
    }
}
```

数据放一类，操作放另一类。

Design A — 使用方式

```
public static void main(String[] args) {  
    PositionA p1 = new PositionA();  
    p1.latitude = 111;  
    p1.longitude = 35;  
    PositionA p2 = new PositionA();  
    p2.latitude = 108;  
    p2.longitude = 36;  
  
    double d = PositionUtility.distance(p1, p2);  
    double h = PositionUtility.heading (p1, p2);  
}
```

欢迎回到 1972 年！ Fortran 在引入 IMSL (International Mathematics and Statistics Library) 时正是这种"数据 + 工具函数"的做法——典型的结构化思维。

Design B — 把数据塞进类里（但仍是 static）

```
public class PositionB {  
    double latitude;  
    double longitude;  
    public static double calculateDistance(double x1, double y1, double x2, double y2) {  
        return Math.sqrt(Math.pow(x2-x1, 2) + Math.pow(y2-y1, 2));  
    }  
    public static double calculateDirection(double x1, double y1, double x2, double y2) {  
        return Math.atan2(y2-y1, x2-x1);  
    }  
    public static void main(String[] args) {  
        double d = calculateDistance(111, 35, 108, 36);  
        double h = calculateDirection(111, 35, 108, 36);  
    }  
}
```

问题：方法虽在类里，但仍是 `static`，参数还是手工传 4 个；类里的 `latitude` / `longitude` 从未被使用。

Design C — 实例方法，但语义残缺

```
public class PositionC {  
    double latitude;  
    double longitude;  
    public double getDistance(double x2, double y2) {  
        return Math.sqrt(Math.pow(x2-latitude, 2) + Math.pow(y2-longitude, 2));  
    }  
    public double getDirection(double x2, double y2) {  
        return Math.atan2(y2-longitude, x2-latitude);  
    }  
}
```

进步：用了实例字段（`this.latitude`）作为"第一个点"。

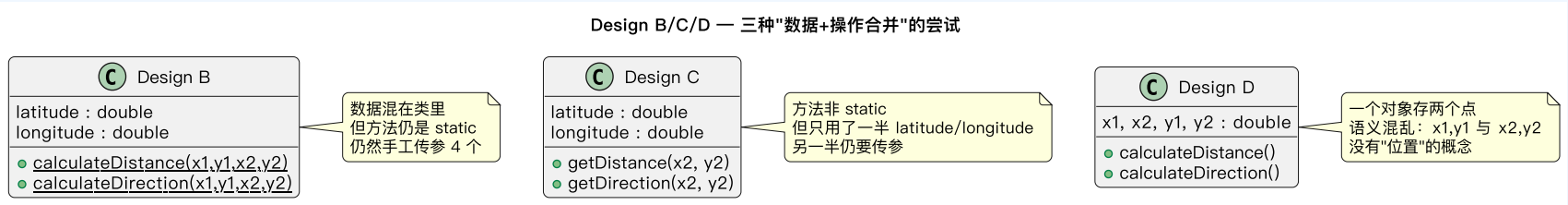
问题：第二个点仍要手工传两个 `double`，没有"两个 Position 之间"的语义。

Design D — 一个对象存两个点（语义混乱）

```
public class PositionD {  
    double x1, x2, y1, y2;  
    public double calculateDistance() { return Math.sqrt(Math.pow(x2-x1, 2) + Math.pow(y2-y1, 2)); }  
    public double calculateDirection() { return Math.atan2(y2-y1, x2-x1); }  
    public static void main(String[] args) {  
        PositionD p1 = new PositionD();  
        p1.x1 = 111; p1.y1 = 35;  
        p1.x2 = 108; p1.y2 = 36;  
        ...  
    }  
}
```

致命问题：一个 `PositionD` 对象同时存了两个点——但它的名字是 "Position"（单数）。对象不再代表现实中的"位置"，而是"一对位置"——语义混乱。

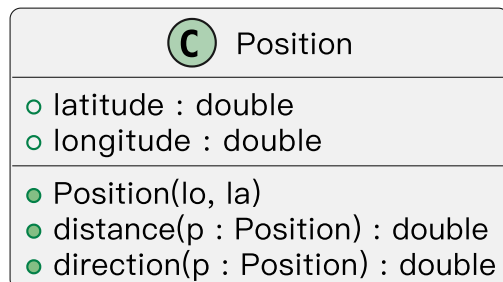
Design B/C/D 对比



这三种设计都试图"把数据放进类里", 但都有缺陷——根本原因: 没有让一个对象代表真实世界中的一个事物 (一个位置点)。

Design I — 一个对象一个点（封装的雏形）

Design I — 一个对象一个点（封装的初步）



一个对象代表一个点
方法天然带 `this`
语义清晰: `p1.distance(p2)`
但字段仍是 `public`

```
public class Position {
    public double longitude, latitude;
    public Position(double lo, double la) { longitude = lo; latitude = la; }
    public double distance (Position p) { return 0.0; }
    public double direction(Position p) { return 0.0; }
    public static void main(String[] args) {
        Position p1 = new Position(111, 35);
        Position p2 = new Position(108, 36);
        double d = p1.distance(p2);    // ← 语义清晰: 从 p1 到 p2 的距离
    }
}
```

Design I — Pros & Cons

Pros 优点

- 语义清晰: `p1.distance(p2)` 表达的是"从 p1 到 p2 的距离"
- 一个对象 = 一个点: 与现实世界对应
- 柯里化效应: `p1.distance(p2)` 隐含地把第一个参数固定下来
- 构造方法: 保证创建时一次性赋值

Cons 缺点

- 字段仍是 `public` ——外部可以任意修改
- 没有数据校验: `new Position(999, -200)` 也能创建非法位置
- 没有边界保护

还差一步: 把字段变成 `private`, 并加入 setter 校验。

Reuse —— Position 类的复用

GPS 系统中，各个点会构成一个图的拓扑结构，表达交通路线。

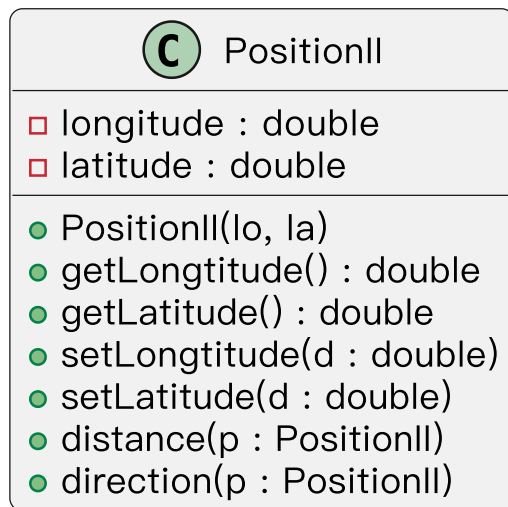
```
// Position 一旦设计好，就可以复用到：
List<Position> route = new ArrayList<>();    // 路径
Map<Position, Building> map = new HashMap<>(); // 地图
Graph<Position> roadNetwork = ...;          // 路网

// 同一个 Position 类支持各种上层应用
```

一个好的封装让一个类成为系统中"一等公民"——可以被无数个上层应用复用。

Design II — 私有字段 + Setter 校验

Design II — 私有字段 + getter/setter (防御性编程)



私有字段 → 外部无法直接改
setter 中校验范围
longitude $\in [0, 180]$
latitude $\in [0, 90]$
这就是封装!

```
public void setLongitude(double d) {
    if (d < 0) longitude = 0;
    else if (d > 180) longitude = 180;
    else longitude = d;
}
```

防御性编程 (Defensive Programming): 把"经度合法范围"这一决策隔离在 setter 里——外部代码不需要关心。

Design II — Pros & Cons

Pros 优点

- **防御性编程** — 非法值被拦截在入口
- **隔离决策** (Isolating the decision): 合法范围的逻辑只存在一处
- **演化友好**: 将来要支持不同坐标系, 只改 setter

Cons 缺点

- 单位是固定的 (默认是度, 不能换千米/英里)
- 几何算法是固定的 (默认是平面距离, 不能换球面/椭圆)
- 一旦需求变化, 仍需大改

下一步: 把"变化的部分"独立出来。

Requirement Change (需求变更)

单位的变化

- Kilometers
- NauticalMiles
- StatueMiles
- Radians

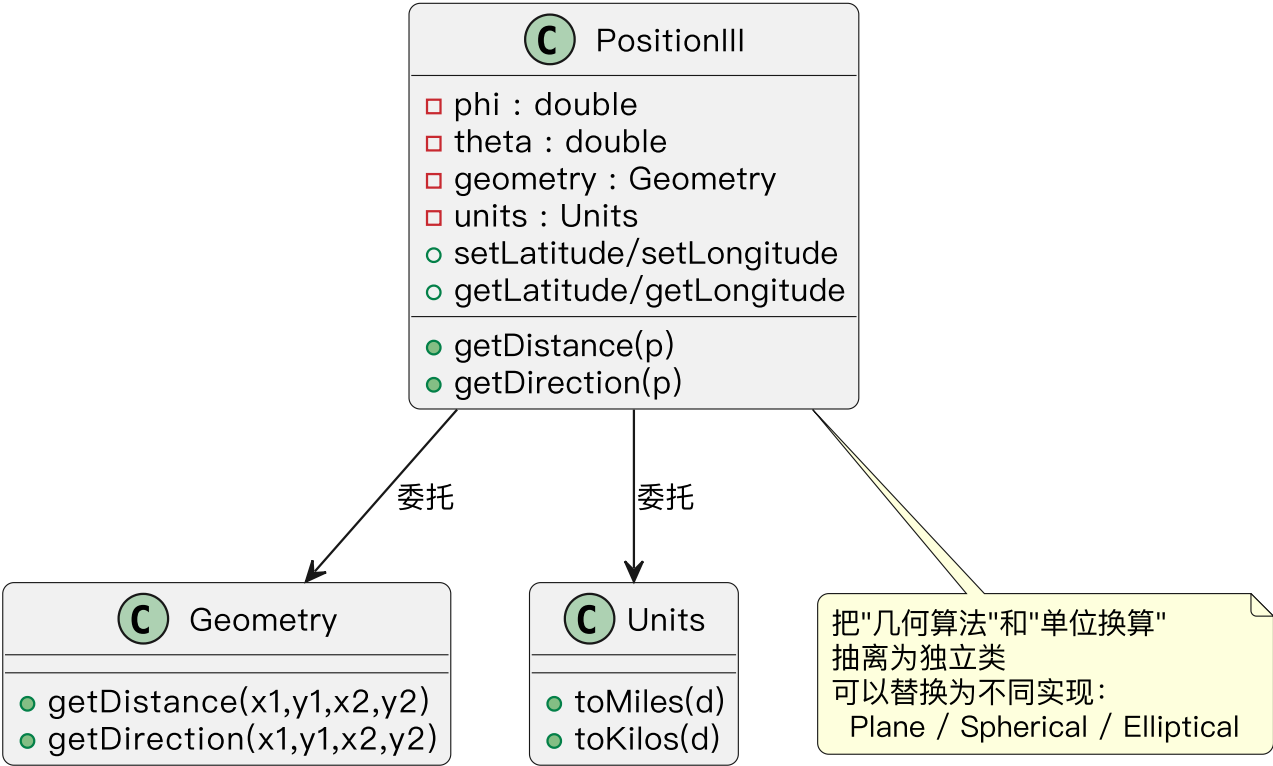
几何模型的变化

- PlaneGeometry 平面几何
- SphericalGeometry 球面几何
- EllipticalGeometry 椭球几何

问题：这两个维度的变化，会让 `distance` / `direction` 的实现急剧膨胀。

Design III — 拆分变化点

Design III — 拆分变化 (Geometry / Units 独立类)



把"几何算法" (Geometry) 和"单位换算" (Units) 抽出来作为独立类—— Position 委托给它们完成具体计算。

Design III — 代码

```
public class PositionIII {
    private double phi, theta;
    private Geometry geometry;
    private Units    units;

    public double getDistance(PositionIII p) {
        return units.toKilos(geometry.getDistance(this.phi, this.theta, p.phi, p.theta));
    }
    public double getDirection(PositionIII p) {
        return geometry.getDirection(this.phi, this.theta, p.phi, p.theta);
    }
}

class Geometry { /* getDistance, getDirection */ }
class Units    { /* toMiles, toKilos */ }
```

Pros: 把潜在的变化点 (geometry / units) 隔离了——以后想换球面几何, 只需替换 `Geometry` 实例。

类比：一个完备的杯子

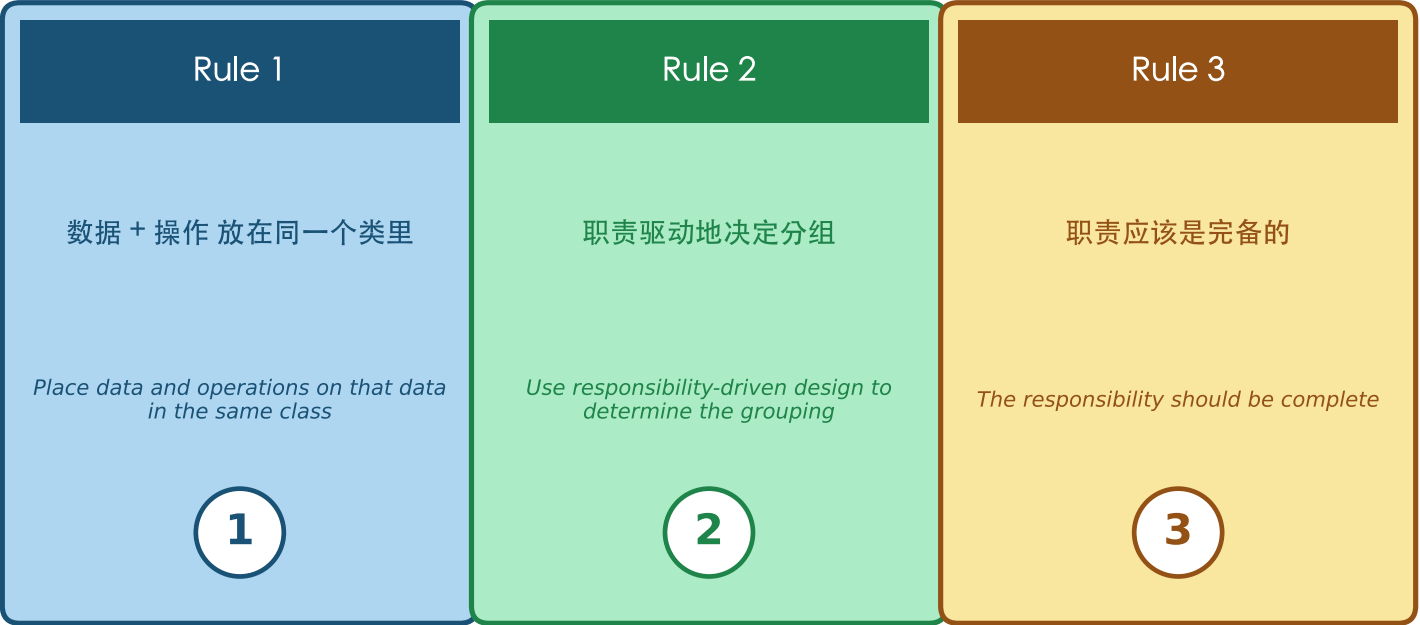
数据 + 操作的封装应该像一个完备的杯子：

- 杯子是**只能加水**的杯子
- 不能从杯子里**倒水出来**
- 而**完备性**：水可以加进去，但不能出去（只读）

这个类比体现了**封装的完备性**：一个类的接口（公开方法）应该提供**完整且自洽**的能力，没有"半截子"的功能。

Encapsulation —— 封装的三条规则

封装的三条规则

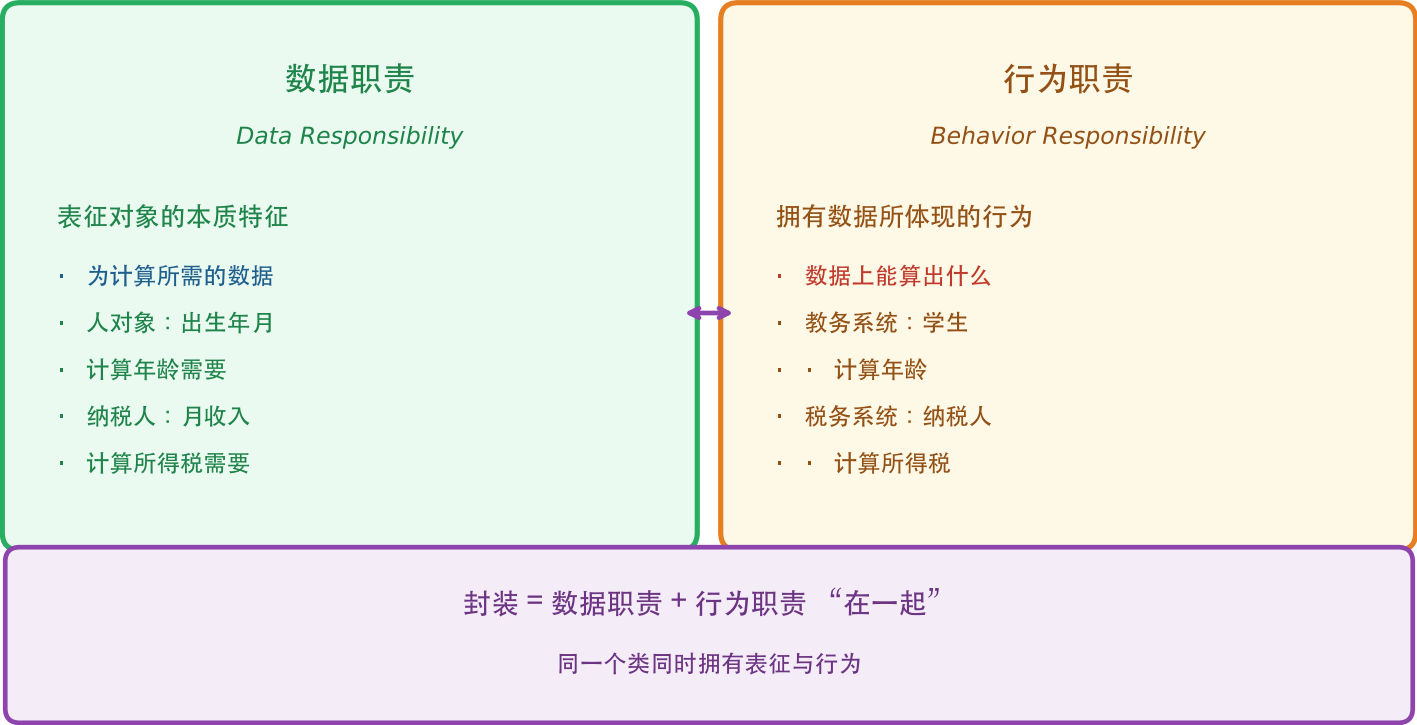


三条规则共同体现：职责 = 数据 + 行为 + 完备

核心：数据 + 操作 + 完备性 = 真正的封装。

类的职责与封装

类的职责与封装



表征对象的本质特征和行为——两者必须在一起。

数据职责的两种来源

① 表征对象的本质

- 人对象：出生年月日（身份）
- 学生对象：学号、姓名、专业
- 图书对象：ISBN、书名、作者

② 为行为（计算）所需

- 教务系统中的人：计算年龄
→ 需要"出生年月日"
- 税务系统中的纳税人：计算所得税
→ 需要"月收入"

同一个"人"在不同系统中的数据职责不同——取决于该系统下的行为需要什么。

行为职责

拥有数据所应该体现的行为——数据上能算出什么？

- 教务系统中的学生：
 - 计算年龄（需要出生年月日）
 - 计算 GPA（需要课程成绩）
- 税务系统中的纳税人：
 - 计算所得税（需要月收入）
 - 生成纳税证明（需要 ID + 累计纳税）

数据决定能做什么行为；行为决定需要什么数据——两者相互定义。

Part 2 — 类

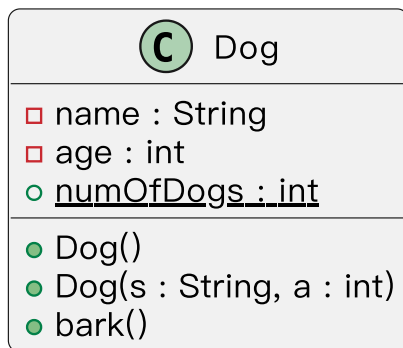
类的成员 · static · final · 构造方法

Outline

- 封装 ✓
- 类
 - 静态方法 / 静态变量 / 静态初始化
 - final 关键字（变量/方法/类）
 - 类的构造方法
- 对象

类的基本构造

类的三部曲：声明 → 创建 → 使用



实例变量：name, age
类变量 (static)：numOfDogs
构造方法：Dog() 和 Dog(s, a)
实例方法：bark()

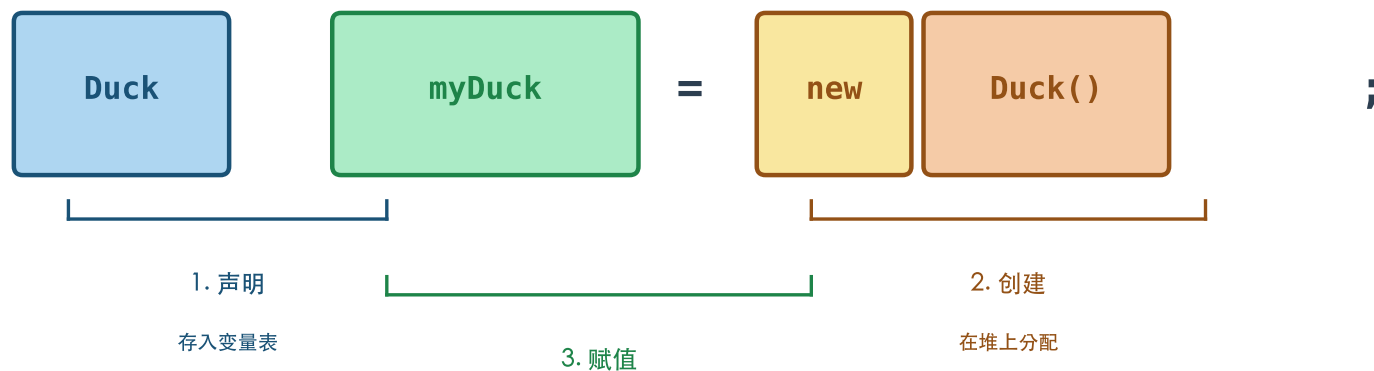
```
class Dog {  
    int size;  
    public Dog();  
    public Dog(int s);  
    public void bark();  
}
```

类包含的成员

- 成员变量 (field): `int size`
- 成员方法 (method): `void bark()`
- 构造方法 (constructor): `Dog(int s)`

对象的三部曲：声明 → 创建 → 赋值

对象的三步话：声明 · 创建 · 赋值



一句话同时完成：变量声明 + 对象创建 + 引用赋值

```
Duck myDuck = new Duck();
```

```
//   ↑      ↑      ↑  
//  变量名  赋值  创建 (new + 构造方法)
```

对象的使用

```
Dog one = new Dog(70);    // 创建

one.size = 70;             // 访问属性 (如果 public)
one.bark();                // 调用方法
```

操作	语法	含义
属性访问	one.size = 70	直接读写字段
方法调用	one.bark()	发送消息给对象

注意： one.size = 70 只在 size 是 public 时合法——否则要用 setter。

Math 类的方法 —— 谁是谁的实例？

```
Math mathObject = new Math();  
// Error: Math() has private access in java.lang.Math
```

为什么不能 `new Math()` ？

```
int x = Math.round(42.2f);    // 直接 Math.xxx 调用  
int y = Math.min(56, 12);  
int z = Math.abs(-343);
```

- `Math` 类的所有方法都是 `static` ——不依赖任何实例状态
- Java 没有"全局函数"——但 `static` 方法本质上就是全局函数
- `Math.round` 等价于 C 的 `round`

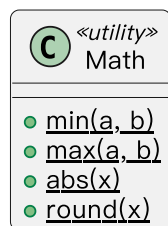
Java 是 OO 语言，但有特殊情况：当行为不依赖对象状态时，让方法直接挂在类上（`static`）。

Static 方法

静态方法 vs 实例方法



实例方法
依赖对象状态 (volume)
player.playSound("a.mp3")



静态方法 (类方法)
无状态依赖
Math.min(42, 36)

关键： `static` 方法不能使用 `instance` 变量——因为它运行时根本没有实例可言。

Static 方法 vs 实例方法

实例方法

```
SoundPlayer player = new SoundPlayer();  
player.playSound(title); // 通过引用调用
```

- 必须先创建对象
- 方法体内 `this` 指向对象
- 可以读写实例变量

静态方法

```
Math.min(42, 36); // 通过类名调用
```

- 无需创建对象
- 方法体内无 `this`
- 不能读写实例变量
- 适合工具函数

使用建议：方法依赖对象状态 → 实例方法；方法只用参数和返回值 → `static`。

Static 变量 —— 类的共享数据

```
public class Dog {  
    public String name;        // 实例变量：每个 Dog 各有一份  
    public int age;  
    public static int numOfDogs; // 静态变量：所有 Dog 共享一份  
  
    public Dog() {  
        name = "Puppy";  
        age = 0;  
        numOfDogs++;           // 每次创建对象，类级别计数 +1  
    }  
}
```

Static Variable: Value is the same for all instances of the class.

所有 `Dog` 对象共享 `numOfDogs` —— 一个改了，所有对象都看到新值。

完整示例：Dog 类（含 static）

```
public class Dog {  
    public String name;  
    public int age;  
    public static int numOfDogs;  
  
    public Dog() { name = "Puppy"; age = 0; numOfDogs++; }  
    public Dog(String s, int a) { name = s; age = a; numOfDogs++; }  
  
    public void bark() {  
        if (age >= 3) System.out.println(name + ":WoWoWo!");  
        else          System.out.println(name + ":Wo!");  
    }  
  
    public static void main(String[] args) {  
        Dog d1 = new Dog("Teddy", 3);  
        Dog d2 = new Dog();           // numOfDogs == 2  
    }  
}
```

Static 变量的初始化

Static variables are initialized when a class is loaded.

两个保证 (Two Guarantees)

1. Static variables are initialized before any object of that class can be created.
 - 任何对象创建之前，类的所有 `static` 变量已经初始化完成
2. Static variables are initialized before any static method of the class runs.
 - 任何 `static` 方法运行之前，类的所有 `static` 变量已经初始化完成

总结：JVM 在首次使用类时加载类，加载过程中完成所有 `static` 变量的初始化。

Static 变量初始化示例

```
public class StaticMethods {  
    static int amount;  
    public StaticMethods(int amount) {  
        this.amount = amount;  
    }  
    public static void eat() {  
        System.out.println("Dog eat " + amount + " bones.");  
    }  
    public static void main(String[] args) {  
        eat(); // amount = 0 (默认)  
        StaticMethods sm = new StaticMethods(2); // amount = 2  
        eat(); // amount = 2  
    }  
}
```

输出: Dog eat 0 bones. → Dog eat 2 bones.

静态初始化顺序 —— 按代码书写顺序

静态初始化顺序 —— 按代码书写顺序

代码

```
class StaticOrder {  
    public static int X = 20;  
    public static int Y = 2 * X;  
    static {  
        X = 30;  
    }  
}  
  
System.out.println(Y); // = ?
```

执行顺序

- 1** **X = 20**
代码第 1 行
- 2** **Y = 2 * X = 40**
代码第 2 行
- 3** **X = 30**
static 块
- **Y = 40**
(Y 已算过, 不重算)

输出: 40 —— 顺序很重要! 交换 static 块位置会得到不同结果

结论: JVM 严格按代码顺序执行 `static` 字段赋值和 `static` 块——交换它们的顺序会得到不同结果。

静态初始化顺序——示例对比

示例 A → 输出 60

```
class Test {  
    public static int X = 10;  
    static { X = 30; }  
    public static int Y = X * 2;  
}  
// X=10 → X=30 → Y=60
```

示例 B → 输出 20

```
class Test {  
    static { X = 30; }    // X 还没声明？编译错  
    public static int X = 10;  
    public static int Y = X * 2;  
}
```

关键洞察

- `static` 字段和 `static` 块按代码出现顺序执行
- 字段会先有默认值 (0/false/null)
- 然后按顺序被赋值/赋值块覆盖

实践建议：不要依赖复杂的静态初始化顺序——把所有依赖都写在一个 `static {}` 块里。

Final Static 变量 —— 真正的常量

final 变量：一旦初始化，永远不能改变。

```
// 命名规范：全大写 + 下划线
public static final double PI = 3.141592653589793;

public class Foo {
    public static final int FOO_X = 3;
}

// 运行时初始化：在 static 块里赋值
public class Bar {
    public static final double BAR_X;
    static { BAR_X = (double) Math.random(); }
}
```

final 关键字的三种用途

final 关键字的三种用途

final 修饰变量

常量

```
final int X = 3;  
// X = 5;  // ERROR
```

final 变量 = 常量

final 修饰方法

不能被重写

```
final void spin() {  
    // no override  
}
```

final 方法 = 冻结行为

final 修饰类

不能有子类

```
public final class String {  
    // no inheritance  
}
```

final 类 = 冻结设计

final 用法 1 —— 修饰变量

一个 final 变量一旦初始化后，永远不能改变。

```
final int X = 3;
// X = 5; // 编译错误: cannot assign a value to final variable X

final double PI = 3.14159;
final Position origin = new Position(0, 0);
// origin = new Position(1, 1); // 错误
origin.setLatitude(10);          // OK! final 限制的是引用，不是对象本身
```

注意: `final Position origin` 限制的是变量 `origin` 不能再指向别的对象——但对象本身的字段仍可修改（只要 setter 允许）。

空白 final (Blank Final)

```
public class BlankFinal {  
    private final int i = 0;          // 有初始值  
    private final int j;              // 空白 final  
    private final Poppet p;          // 空白 final 引用  
  
    // 空白 final 必须在构造方法中初始化  
    public BlankFinal() { j = 1; p = new Poppet(1); }  
    public BlankFinal(int x) { j = x; p = new Poppet(x); }  
}
```

空白 final: 声明时不赋初值，但**必须**在构造方法中初始化。

好处: 对象创建时一次性确定值，之后不可变 —— 不可变对象的实现关键。

final 参数

```
class Gizmo { public void spin() {} }

public class FinalArguments {
    void with (final Gizmo g) {
        // g = new Gizmo(); // 错! g 是 final
        g.spin();           // OK: 调用方法不算修改引用
    }
    void without(Gizmo g) {
        g = new Gizmo();    // OK: g 不是 final
    }

    int g(final int i) { return i + 1; } // 只能读 i, 不能改
}
```

用 `final` 参数表明: 这个参数在方法体内不会被重新赋值。

final 用法 2 —— 修饰方法

使用 final 方法的两个原因

原因 1：设计上的考虑

把方法**锁定**，预防任何继承类修改它的意义。
确保在继承中方法行为保持不变，并且**不会被重写**。

```
public class Animal {  
    public final String getId() {  
        return idGenerator.next();  
    }  
    // 子类不能 override  
}
```

原因 2：效率优化

早期 JVM 的优化：将一个方法指明为 final，编译器可以将该方法的所有调用都转为内嵌调用（inline）。

```
public final void smallHelper() {  
    return x * 2;  
}
```

现代 JIT 已经会自动 inline，这条原因如今较少使用。

final 用法 3 —— 修饰类

```
public final class String {  
    // String 是 final 类, 没有任何子类  
}  
  
// public class MyString extends String { }    // 编译错误
```

设计意图: `String` 类的设计者希望它的语义永远不变——不能被子类化扭曲。

常见的 final 类

- `String`、`Integer`、`Double` 等所有包装类
- 工具类（如设计上不希望被继承的 helper）

`final` 类 = 设计已经凝固, 不允许扩展。

类的构造方法

```
public class Mushroom {  
    public Mushroom() {  
        // 在对象被赋给引用之前运行  
        // 给你机会"步入 new 的中间"  
        // 初始化对象的状态  
    }  
}
```

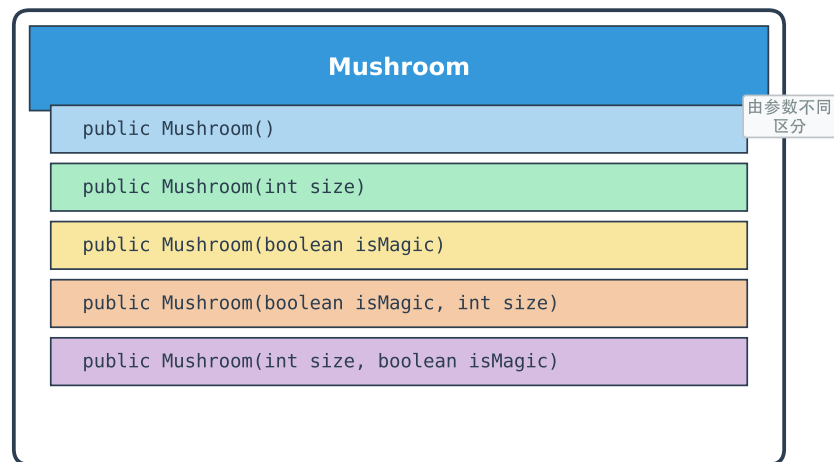
关键特性

- 没有返回类型（连 `void` 都没有）
- 方法名必须和类名相同
- 在 `new` 表达式中被自动调用
- 给你"step into the middle of new"的机会

构造方法的唯一目的：初始化对象的状态。

构造方法的重载

构造方法重载 — 多种创建方式



重载仅看参数列表（个数、类型、顺序），与返回类型无关

```
public Mushroom()  
    { /*...*/ }  
  
public Mushroom(int size)  
    { /*...*/ }  
  
public Mushroom(boolean isMagic)  
    { /*...*/ }  
  
public Mushroom(boolean isMagic, int size)  
    { /*...*/ }  
  
// 顺序不同也算重载  
public Mushroom(int size, boolean isMagic)  
    { /*...*/ }
```

重载：同名、不同参数列表（类型 / 个数 / 顺序）。

构造方法的重要规则

规则 1：默认无参构造

如果你没写构造方法，编译器会帮你创建一个无参的：

```
public Mushroom() {}
```

规则 2：写了有参 → 必须自写无参

如果你写了有参数的构造方法：

```
public Mushroom(int size) { ... }
```

编译器不再帮你创建无参的——若代码里有 `new Mushroom()`，会编译错。

解决：自己也写一个无参

```
public Mushroom() { ... }  
public Mushroom(int size) { ... }
```

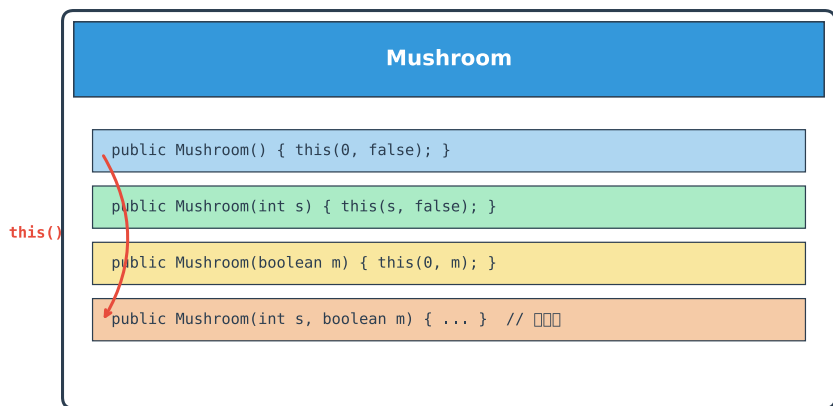
重载判定规则

是	否
参数类型不同	仅参数名不同
参数个数不同	仅返回类型不同
参数顺序不同	

与方法重载规则完全一致。

构造方法之间的 `this()` 调用

构造方法互相调用 — `this()` 接力棒模式



优点：避免代码重复，初始化逻辑集中在最底层构造方法，`this()` 必须是构造方法的第一句

```
public class Mushroom {  
    private int size;  
    private boolean isMagic;  
  
    public Mushroom() {  
        this(10, false);    // 调主构造  
    }  
    public Mushroom(int size) {  
        this(size, false);  // 调主构造  
    }  
    public Mushroom(int size, boolean isMagic) {  
        this.size = size;    // ★ 主构造  
        this.isMagic = isMagic;  
    }  
}
```

三条铁律

1. `this(...)` 必须是第一行
2. 不能形成循环调用
3. `this(...)` 与 `super(...)` 二选一

意图：把"真正的初始化"集中到一个主构造方法，其他通过 `this()` 委托——避免重复代码。

Part 3 — 对象

初始化 · 引用 · 生存期 · 垃圾回收

对象初始化的初步

The variable are initialized before any methods can be called, even the constructor.

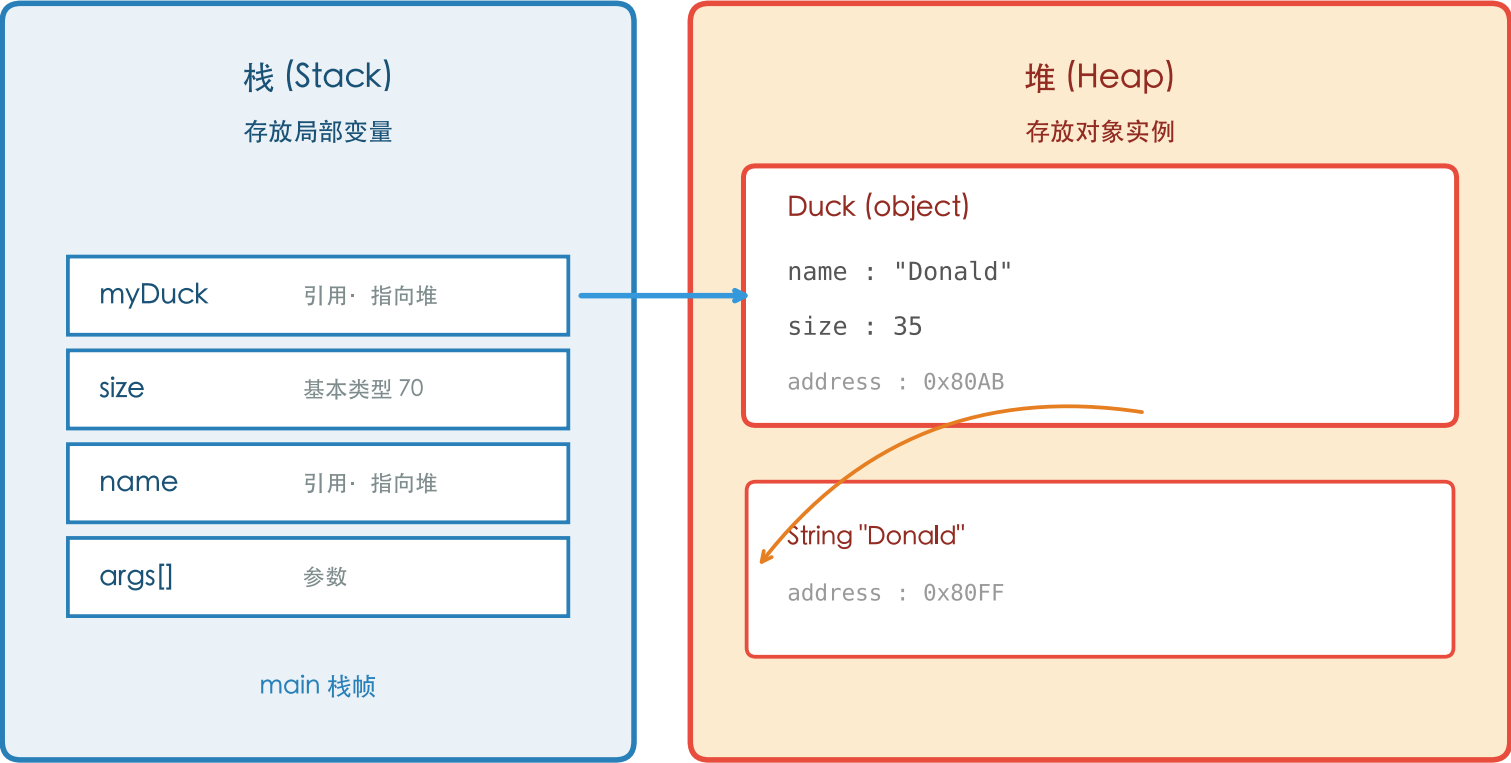
初始化顺序保证

优先级	内容
1	Static data initialized first
2	Then non-static (instance) data
3	同类型内部按代码书写顺序执行

关键：构造方法不是最先执行的——它运行之前，所有 `static` 变量、所有 `static` 块、所有实例变量的默认初始化都已完成。

引用与对象 —— Stack vs Heap

变量在栈、对象在堆



引用变量存于栈，实际对象在堆上分配

核心：变量在栈上分配（生命周期短），对象在堆上分配（生命周期由 GC 管理）。

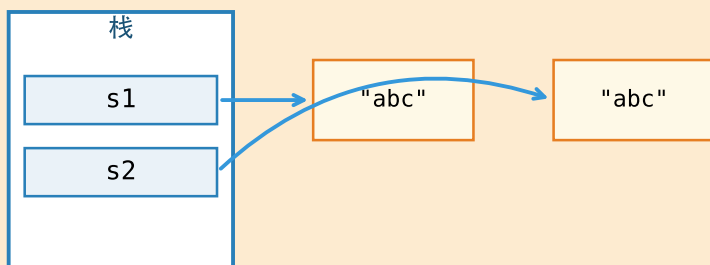
== vs equals

== vs equals — 引用相同 vs 内容相同

== 比较引用

(地址是否相同)

```
String s1 = new String("abc");  
String s2 = new String("abc");  
if (s1 == s2) // false △
```



两个不同的堆对象 → 地址不同 → false

equals 比较内容

(字符串是否相同)

```
if (s1.equals(s2)) // true ✓
```

```
String s3 = "abc";  
String s4 = "abc";  
if (s3 == s4) // true ✓
```

字符串字面量进入常量池

内容相同的字面量 → 复用同一个对象

s3 和 s4 指向同一个字符串

== vs equals —— 完整代码示例

```
String s1 = new String("abc");
String s2 = new String("abc");
if (s1 == s2)          System.out.println("yes"); else System.out.println("no"); // no
if (s1.equals(s2))     System.out.println("yes"); else System.out.println("no"); // yes

String s3 = "abc";     // 字面量, 进字符串常量池
String s4 = "abc";
if (s3 == s4)          System.out.println("yes"); // yes
```

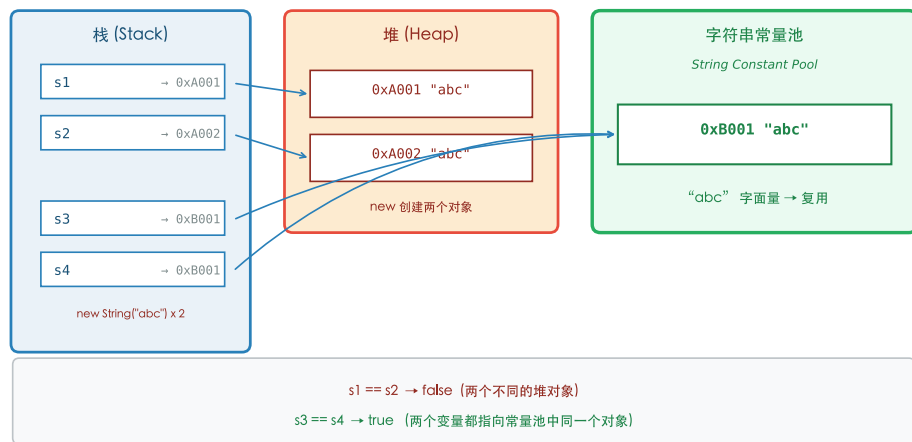
```
Integer integer = new Integer(100);
System.out.println(integer.intValue()); // 自动拆箱
ArrayList<Integer> scoresList = new ArrayList<Integer>();
// java.lang.Object is the root class
```

铁律:

- 基本类型 用 `==` 比较
- 引用类型 (String/对象等) 用 `equals()` 比较内容

字符串常量池 (String Pool)

字符串常量池 == 与 equals 的底层机制



两种创建方式

```
String a = "hello";           // 字面量 → 池
String b = "hello";           // 复用池中对象
String c = new String("hello"); // 堆上新建
```

```
System.out.println(a == b);    // true
System.out.println(a == c);    // false
System.out.println(a.equals(c)); // true
```

为什么有常量池?

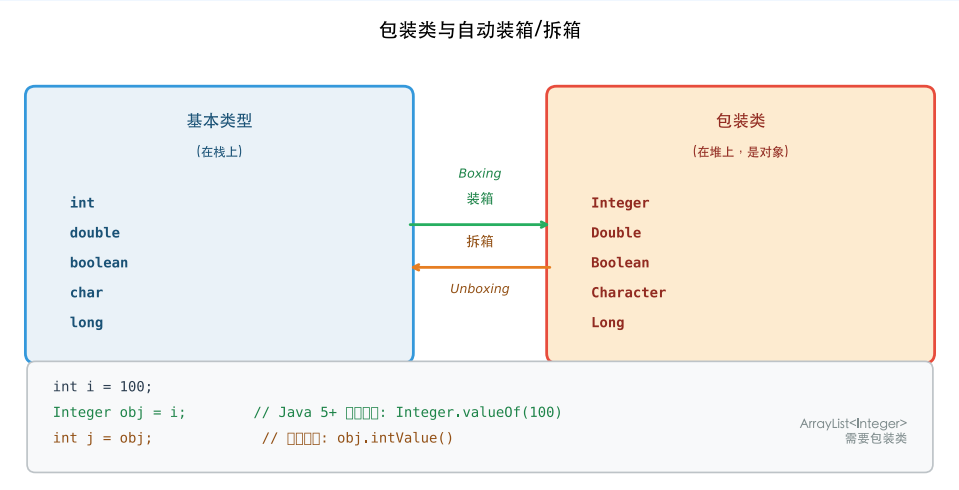
- 节省内存: 相同字面量共享同一对象
- 加快比较: 可用 `==` 直接比较地址
- `String` 是不可变的 (`final` 类) —— 共享是安全的

`intern()` 方法

```
String c = new String("hello").intern();
System.out.println(a == c); // true
```

`intern()`: 把字符串放入常量池或返回池中已有引用——手动入池。

包装类与自动装箱拆箱



8 种基本类型的包装类

基本类型	包装类
int	Integer
long	Long
double	Double
boolean	Boolean
char	Character
byte / short / float	Byte / Short / Float

装箱 / 拆箱

```
Integer x = 100; // 装箱: Integer.valueOf(100)
int y = x;       // 拆箱: x.intValue()

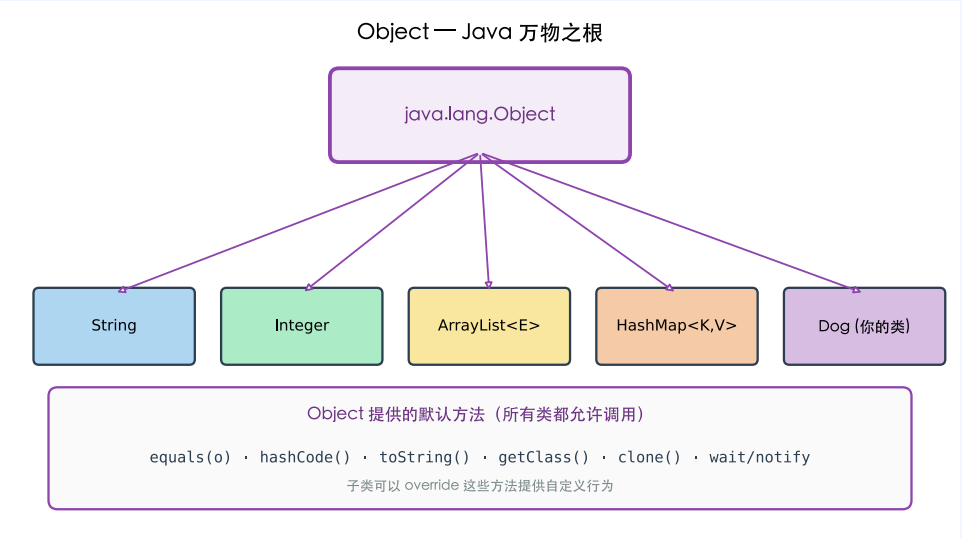
List<Integer> list = new ArrayList<>();
list.add(1);      // 自动装箱
```

Integer 缓存陷阱

```
Integer a = 127, b = 127;
Integer c = 128, d = 128;
System.out.println(a == b); // true  -128~127 缓存
System.out.println(c == d); // false 超出缓存
```

铁律：包装类比较永远用 `equals()`。

Object —— Java 万物之根



java.lang.Object 的核心方法

方法	默认实现	何时重写
equals(Object)	== 引用相等	比较内容时
hashCode()	地址相关	与 equals 成对
toString()	类名@十六进制	调试 / 打印
getClass()	Class 对象	反射
wait/notify	—	多线程同步

所有类隐式继承 Object

```
public class Dog { /* ... */ }
// 等价于
public class Dog extends Object { /* ... */ }
```

多态的根本

```
Object[] anything = new Object[3];
anything[0] = "hello";
anything[1] = Integer.valueOf(42);
anything[2] = new Dog();
```

任何对象都可以被当作 Object 引用——这是 Java 容器（早期 ArrayList 装 Object）和反射的根基。

变量的生存期

局部变量（栈）

方法执行的时间段

```
void f() {  
    int x = 10;           // 进入方法, x 诞生  
    String s = "abc";  
    // ... 使用 x, s ...  
}                          // 离开方法, x、s 消亡
```

成员变量（堆）

对象存活的时间段

```
class Dog {  
    int age;              // 与所属 Dog 对象同存亡  
}  
  
Dog d = new Dog();       // d 引用的对象诞生  
d = null;                 // 该对象不再可达 → 等待 GC
```

关键差异：局部变量随方法栈帧消亡，成员变量随对象消亡（GC 决定时机）。

垃圾回收的触发时机

```
void go() {  
    Life z = new Life();  
}  
// ① 方法执行完毕，局部变量 z 消失，源对象可被回收
```

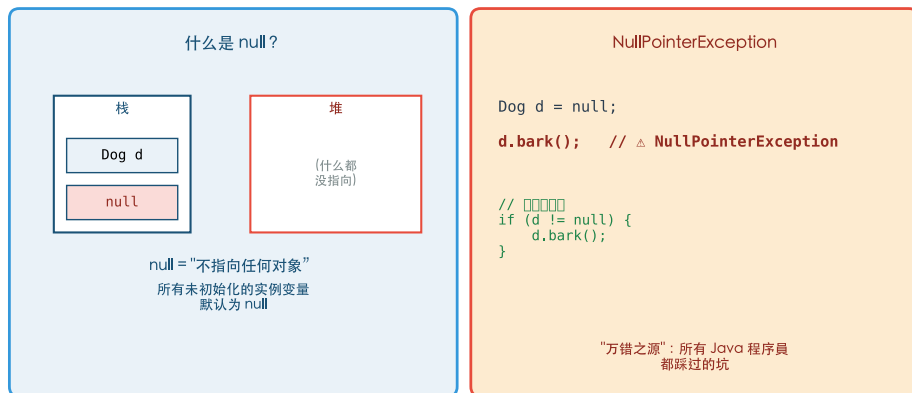
```
void go() {  
    Life z = new Life();  
    z = new Life(); // ② z 指向另一对象，原对象可被回收  
}
```

```
void go() {  
    Life z = new Life();  
    z = null; // ③ z 指向 null，原对象可被回收  
}
```

共同点：没有任何引用指向某个对象时，该对象就可以被 GC 回收。

null 与 NullPointerException

null 与 NullPointerException



null 是什么?

- `null` 表示引用不指向任何对象
- 引用类型变量的默认值是 `null`
- 基本类型没有 `null`

```
String s = null;  
Dog d;           // 成员默认 null  
int x;           // 成员默认 0
```

防御性编程

```
// ① 显式判空  
if (s != null) s.length();  
  
// ② Objects.requireNonNull  
this.name = Objects.requireNonNull(name, "name 不能为 null");  
  
// ③ Java 8+ Optional  
Optional.ofNullable(s).ifPresent(System.out::println);  
  
// ④ 字符串字面量在前 — 永远不 NPE  
if ("admin".equals(role)) { ... }
```

设计取舍: `null` 看似无成本, 但成倍增加防御代码。
`Optional` 让“可能不存在”成为类型层面的明示。

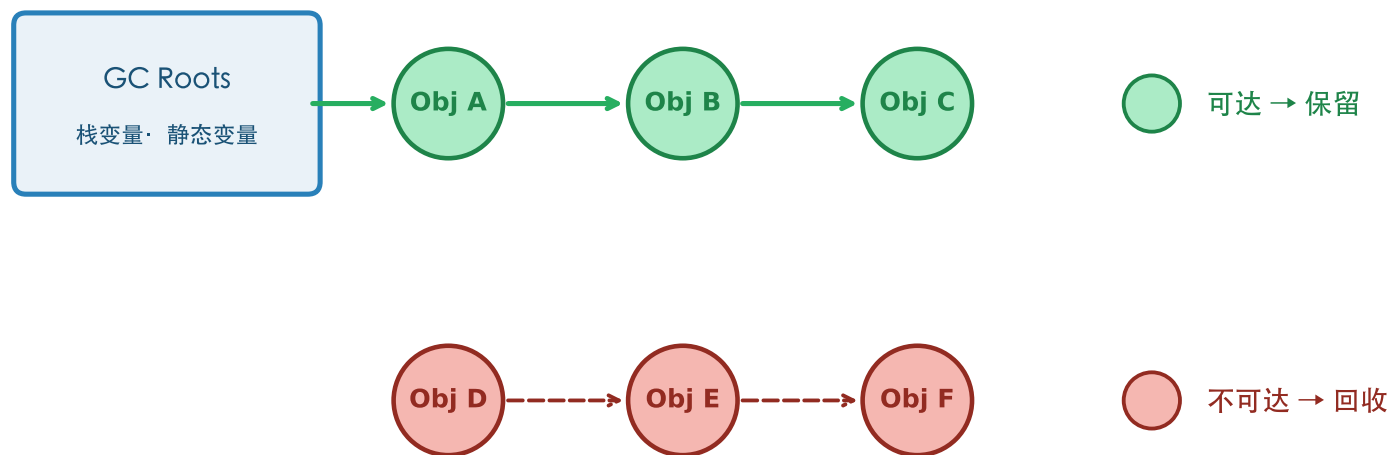
垃圾回收器的三件任务

Java 的垃圾回收器要负责完成 3 件任务：

任务	说明
① 分配内存	在堆上为新对象划分空间
② 确保被引用的对象不被错误回收	通过可达性分析
③ 回收不再被引用的对象的内存空间	释放空间供后续使用

可达性分析

垃圾回收—可达性分析



从 GC Roots 出发，可以递归到达的对象保留，其余全部回收

即使 D-E-F 互相引用（环），也会被回收

核心：从 GC Roots（栈变量、静态变量等）出发，能到达的对象保留，到达不了的全部回收（即使形成"环引用"）。

为什么 GC 是个复杂操作？

垃圾回收是一个复杂且耗时的操作。如果 JVM 花费过多时间在垃圾回收上，则势必会影响应用的运行性能。

关键事实

- 一般情况下，垃圾回收时**整个应用执行被暂时中止**（stop-the-world）
- 因为 GC 需要更新所有对象引用的**实际内存地址**
- 不同硬件平台支持不同的垃圾回收方式：
 - 多 CPU → 可以**并发**回收
 - 单 CPU → 只能**串行**进行
- 不同应用对 GC 的期望不同：
 - 服务器端：希望 GC 总时间最短
 - 用户交互应用：希望 GC 停顿间隔最长

JVM 提供了多种 GC 算法和参数，可根据需要调优。

Java 分代回收

JVM 垃圾回收 — 分代回收



理论依据：大部分对象是“朝生夕死的”，针对不同世代用不同算法能提高效率

内存中的区域被划分成不同的世代，对象根据其存活的时间被保存在对应世代的区域中。

分代的理论依据

一般实现：3 个世代

世代	存放	算法特点
年轻 (Young)	新创建的对象	大部分会很快死亡
年老 (Old)	存活很久的对象	长寿命，回收较少
永久 (Permanent)	类信息、常量池	一般不回收

设计出发点

- 内存的分配**首先发生在年轻世代**
- 当对象**存活时间足够长**时 → 复制到年老世代
- 不同世代用**不同算法**：针对其生命周期特性做优化

统计规律：应用中大部分对象的存活时间都很短——比如局部变量。

年轻世代的内存结构

年轻世代的内存区域被进一步划分成：

- 伊甸园 (Eden) — 进行内存分配的地方
- 两个存活区 (Survivor space) — 始终有一个是空的

工作流程

1. 新对象总是分配在 **Eden** (连续的空闲内存, 分配非常快)
2. GC 时, Eden + 一个 Survivor 中存活的对象 → 复制到**另一个空 Survivor**
3. 之前的 Eden 和原 Survivor 直接清空 ("**复制收集**")
4. 下次 GC, 两个 Survivor 角色互换

年轻世代区域较小, 且**大部分对象都已不再存活**——查找存活对象的效率较高。

年老世代的算法 —— Mark-Sweep-Compact

对于年老和永久世代，采用的是标记-清除-压缩算法。

步骤	操作
① Mark（标记）	找出当前还存活的对象，进行标记
② Sweep（清除）	遍历整个内存区域，找出需要回收的区域
③ Compact（压缩）	把存活对象的内存移动到整个区域的一端，使另一端是连续的空闲区域，便于内存分配和复制

目标：避免内存碎片化——保证总有连续的大块空闲内存可用。

Outline 回顾

封装 ✓

- 5 种设计的演进
- 封装三规则：数据 + 操作 + 完备
- 数据职责 + 行为职责

类 ✓

- 成员变量 / 成员方法 / 构造方法
- static 变量 / static 方法
- 静态初始化顺序
- final 关键字（变量/方法/类/参数）
- 构造方法重载

对象 ✓

- 对象初始化顺序
- 引用：Stack vs Heap
- 引用比较：== vs equals
- 生存期：局部变量 vs 成员变量
- 垃圾回收：可达性分析
- 分代回收（Young / Old / Permanent）
- 年轻世代：Eden + Survivor × 2
- 年老世代：Mark–Sweep–Compact

本讲总结

面向对象编程 II – 封装

封装的本质

数据 + 操作 + 完备性 = 真正的封装

从 Position A → I → II → III 的演进

类的成员

实例变量 / static 变量

实例方法 / static 方法

构造方法 / 构造方法重载

关键字

static – 类级别共享

final – 不可变

对象生命周期

创建 → 使用 → 不可达 → GC 回收

分代回收: Young + Old + Permanent

下一讲：面向对象编程 III — 继承与多态

面向对象编程 II — 封装

谢谢聆听 · 有问题随时提问